```python
# ── STEP 0: Mount Google Drive ──────────────────────────────
from google.colab import drive
drive.mount('/content/drive')

# ── IMPORTS ─────────────────────────────────────────────────
import numpy as np
import pandas as pd
from sklearn.preprocessing import MinMaxScaler
from sklearn.cluster import KMeans
import torch
import torch.nn as nn
```

```
Mounted at /content/drive
```

```python
# TASK 1: Load and Clean Data
# ============================================================

# ── Load dataset from Google Drive ──────────────────────────
FILE_PATH = "/content/drive/MyDrive/online_retail_II.xlsx"
# If your file has a different name, change it above ☝

df1 = pd.read_excel(FILE_PATH, sheet_name="Year 2009-2010")
df2 = pd.read_excel(FILE_PATH, sheet_name="Year 2010-2011")
df = pd.concat([df1, df2], ignore_index=True)

print("□ Dataset loaded successfully!")
print("Original shape:", df.shape)
print("\nColumn names:", df.columns.tolist())
print("\nFirst 3 rows:\n", df.head(3))
print("\nMissing values:\n", df.isnull().sum())

# ── Clean the data ──────────────────────────────────────────
# Remove rows missing Customer ID or Description
df.dropna(subset=["Customer ID", "Description"], inplace=True)

# Remove cancelled invoices (start with 'C')
df = df[~df["Invoice"].astype(str).str.startswith("C")]

# Remove rows with zero or negative Quantity/Price
df = df[(df["Quantity"] > 0) & (df["Price"] > 0)]

# Convert Customer ID to integer
df["Customer ID"] = df["Customer ID"].astype(int)

print("\n□ Data cleaned!")
print("Cleaned shape:", df.shape)
```

```
□ Dataset loaded successfully!
Original shape: (1067371, 8)

Column names: ['Invoice', 'StockCode', 'Description', 'Quantity',
```

```
'InvoiceDate', 'Price', 'Customer ID', 'Country']

First 3 rows:
   Invoice StockCode                          Description  Quantity  \
0  489434     85048  15CM CHRISTMAS GLASS BALL 20 LIGHTS        12
1  489434    79323P                   PINK CHERRY LIGHTS        12
2  489434    79323W                  WHITE CHERRY LIGHTS        12

          InvoiceDate  Price  Customer ID         Country
0 2009-12-01 07:45:00   6.95      13085.0  United Kingdom
1 2009-12-01 07:45:00   6.75      13085.0  United Kingdom
2 2009-12-01 07:45:00   6.75      13085.0  United Kingdom

Missing values:
 Invoice             0
StockCode           0
Description      4382
Quantity            0
InvoiceDate         0
Price               0
Customer ID    243007
Country             0
dtype: int64

 Data cleaned!
Cleaned shape: (805549, 8)
```

```python
# TASK 2: Preprocess Data
# ============================================================

# Create TotalPrice feature
df["TotalPrice"] = df["Quantity"] * df["Price"]

# Extract month and year from InvoiceDate
df["InvoiceDate"] = pd.to_datetime(df["InvoiceDate"])
df["YearMonth"] = df["InvoiceDate"].dt.to_period("M")

# Encode Country as category codes
df["CountryCode"] = df["Country"].astype("category").cat.codes

# Scale numerical features: Quantity, Price, TotalPrice
scaler = MinMaxScaler()
df[["Quantity_scaled", "Price_scaled", "TotalPrice_scaled"]] =
scaler.fit_transform(
    df[["Quantity", "Price", "TotalPrice"]]
)

print("\nPreprocessed Data Sample:")
print(df[["Customer ID", "StockCode", "Quantity_scaled",
```

```
"Price_scaled",
          "TotalPrice_scaled", "CountryCode"]].head())
```

```
Preprocessed Data Sample:
   Customer ID StockCode  Quantity_scaled  Price_scaled
TotalPrice_scaled  \
0        13085     85048         0.000136      0.000634
0.000495
1        13085    79323P         0.000136      0.000616
0.000481
2        13085    79323W         0.000136      0.000616
0.000481
3        13085     22041         0.000580      0.000192
0.000598
4        13085     21232         0.000284      0.000114
0.000178

   CountryCode
0           38
1           38
2           38
3           38
4           38
```

```
# TASK 3: Transform Data into Binary Format
# ============================================================
# Each row = one customer
# Each column = one product (StockCode)
# Value = 1 if customer purchased that product, else 0

# Use top N products to keep matrix manageable
TOP_N = 500
top_products =
df["StockCode"].value_counts().head(TOP_N).index.tolist()
df_top = df[df["StockCode"].isin(top_products)]

# Pivot: customers x products, binary (purchased=1, not=0)
binary_matrix = df_top.groupby(["Customer ID", "StockCode"])
["Quantity"].sum()
binary_matrix = binary_matrix.unstack(fill_value=0)
binary_matrix = (binary_matrix > 0).astype(float)

print("\nBinary Matrix Shape:", binary_matrix.shape)
print("Sample (first 5 customers, first 10 products):")
print(binary_matrix.iloc[:5, :10])

# Convert to tensor for PyTorch
data_tensor = torch.FloatTensor(binary_matrix.values)
```

```
# Train/test split (80/20)
train_size = int(0.8 * len(data_tensor))
train_data = data_tensor[:train_size]
test_data  = data_tensor[train_size:]

print(f"\nTrain size: {train_data.shape[0]}, Test size:
{test_data.shape[0]}")
```

```
Binary Matrix Shape: (5724, 500)
Sample (first 5 customers, first 10 products):
StockCode    15036  20675  20676  20677  20679  20682  20685  20711
20712  \
Customer ID

12346          0.0    0.0    0.0    0.0    1.0    1.0    1.0    0.0
0.0
12347          0.0    0.0    0.0    0.0    0.0    0.0    0.0    0.0
1.0
12348          0.0    0.0    0.0    0.0    0.0    0.0    0.0    0.0
0.0
12349          0.0    0.0    0.0    0.0    0.0    0.0    1.0    0.0
0.0
12350          0.0    0.0    0.0    0.0    0.0    0.0    0.0    0.0
0.0

StockCode    20713
Customer ID
12346          0.0
12347          0.0
12348          0.0
12349          0.0
12350          0.0

Train size: 4579, Test size: 1145
```

```
# TASK 4: Train the Boltzmann Machine (RBM)
# ============================================================

class RBM(nn.Module):
    """Restricted Boltzmann Machine using Contrastive Divergence (CD-
k)"""

    def __init__(self, n_visible, n_hidden):
        super(RBM, self).__init__()
        self.W  = nn.Parameter(torch.randn(n_hidden, n_visible) *
0.01)
        self.bh = nn.Parameter(torch.zeros(n_hidden))   # hidden bias
        self.bv = nn.Parameter(torch.zeros(n_visible))  # visible bias
```

```
    def sample_hidden(self, v):
        """Compute P(h=1|v) and sample"""
        prob_h = torch.sigmoid(v @ self.W.t() + self.bh)
        return prob_h, torch.bernoulli(prob_h)

    def sample_visible(self, h):
        """Compute P(v=1|h) and sample"""
        prob_v = torch.sigmoid(h @ self.W + self.bv)
        return prob_v, torch.bernoulli(prob_v)

    def contrastive_divergence(self, v0, k=1):
        """CD-k: k steps of Gibbs sampling"""
        # Positive phase
        prob_h0, h0 = self.sample_hidden(v0)

        # Negative phase (k Gibbs steps)
        vk, hk = v0.clone(), h0.clone()
        for _ in range(k):
            prob_vk, vk = self.sample_visible(hk)
            prob_hk, hk = self.sample_hidden(vk)

        # Gradients (positive - negative associations)
        positive_grad = prob_h0.t() @ v0
        negative_grad = prob_hk.t() @ vk

        return positive_grad - negative_grad, vk, prob_hk

    def free_energy(self, v):
        """Compute free energy for monitoring training"""
        vbias_term = v @ self.bv
        hidden_term = torch.log(1 + torch.exp(v @ self.W.t() +
self.bh)).sum(dim=1)
        return (-vbias_term - hidden_term).mean()

# ── Hyperparameters ──────────────────────────────────────────
N_VISIBLE = binary_matrix.shape[1]   # number of products
N_HIDDEN  = 100                       # hidden units (latent features)
EPOCHS    = 20
BATCH_SIZE = 64
LR        = 0.01
K         = 1                         # CD-k steps

rbm = RBM(N_VISIBLE, N_HIDDEN)
optimizer = torch.optim.SGD(rbm.parameters(), lr=LR)

print("\n--- Training RBM ---")
print(f"Visible units: {N_VISIBLE}, Hidden units: {N_HIDDEN}")
print(f"Epochs: {EPOCHS}, Batch size: {BATCH_SIZE}, LR: {LR}\n")

train_losses = []
```

```
for epoch in range(1, EPOCHS + 1):
    epoch_loss = 0.0
    # Shuffle training data each epoch
    perm = torch.randperm(train_data.shape[0])
    train_shuffled = train_data[perm]

    for i in range(0, train_data.shape[0], BATCH_SIZE):
        batch = train_shuffled[i: i + BATCH_SIZE]

        optimizer.zero_grad()

        # CD-k gradients
        grad, vk, prob_hk = rbm.contrastive_divergence(batch, k=K)

        # Manually assign gradients (CD is not automatic diff)
        rbm.W.grad  = -grad / batch.shape[0]
        rbm.bv.grad = -(batch - vk).mean(dim=0)
        rbm.bh.grad = -(rbm.sample_hidden(batch)[0] -
prob_hk).mean(dim=0)

        optimizer.step()

        # Reconstruction error as proxy loss
        _, h   = rbm.sample_hidden(batch)
        v_recon, _ = rbm.sample_visible(h)
        loss = torch.mean((batch - v_recon) ** 2)
        epoch_loss += loss.item()

    avg_loss = epoch_loss / (train_data.shape[0] // BATCH_SIZE)
    train_losses.append(avg_loss)

    if epoch % 5 == 0 or epoch == 1:
        print(f"Epoch [{epoch:>2}/{EPOCHS}]  Reconstruction Loss:
{avg_loss:.4f}")
```

```
--- Training RBM ---
Visible units: 500, Hidden units: 100
Epochs: 20, Batch size: 64, LR: 0.01

Epoch [ 1/20]  Reconstruction Loss: 0.1406
Epoch [ 5/20]  Reconstruction Loss: 0.0659
Epoch [10/20]  Reconstruction Loss: 0.0628
Epoch [15/20]  Reconstruction Loss: 0.0621
Epoch [20/20]  Reconstruction Loss: 0.0617
```

```
# ── Evaluation on Test Set ──────────────────────────────────
_, h_test  = rbm.sample_hidden(test_data)
v_recon_test, _ = rbm.sample_visible(h_test)
test_loss = torch.mean((test_data - v_recon_test) ** 2).item()
```

```python
print(f"\nTest Reconstruction Loss: {test_loss:.4f}")

# ── Extract Hidden Representations (Customer Embeddings) ────
with torch.no_grad():
    prob_h_train, _ = rbm.sample_hidden(train_data)
    prob_h_test,  _ = rbm.sample_hidden(test_data)

print("\nHidden representation (customer embedding) shape:",
prob_h_train.shape)
print("Each customer is now represented as a", N_HIDDEN, "-dimensional
latent vector.")

# ── Simple Customer Clustering using embeddings ──────────────
from sklearn.cluster import KMeans

embeddings = prob_h_train.detach().numpy()
kmeans = KMeans(n_clusters=5, random_state=42, n_init=10)
clusters = kmeans.fit_predict(embeddings)

# Attach cluster labels back to customer IDs
customer_ids_train = binary_matrix.index[:train_size].tolist()
cluster_df = pd.DataFrame({
    "Customer ID": customer_ids_train,
    "Cluster": clusters
})

print("\nCustomer Segment Distribution:")
print(cluster_df["Cluster"].value_counts().sort_index())
print("\nSample Customer Segments:")
print(cluster_df.head(10))
```

```
Test Reconstruction Loss: 0.0565

Hidden representation (customer embedding) shape: torch.Size([4579,
100])
Each customer is now represented as a 100 -dimensional latent vector.

Customer Segment Distribution:
Cluster
0     606
1    1009
2     629
3    1835
4     500
Name: count, dtype: int64

Sample Customer Segments:
   Customer ID  Cluster
0        12346        1
```

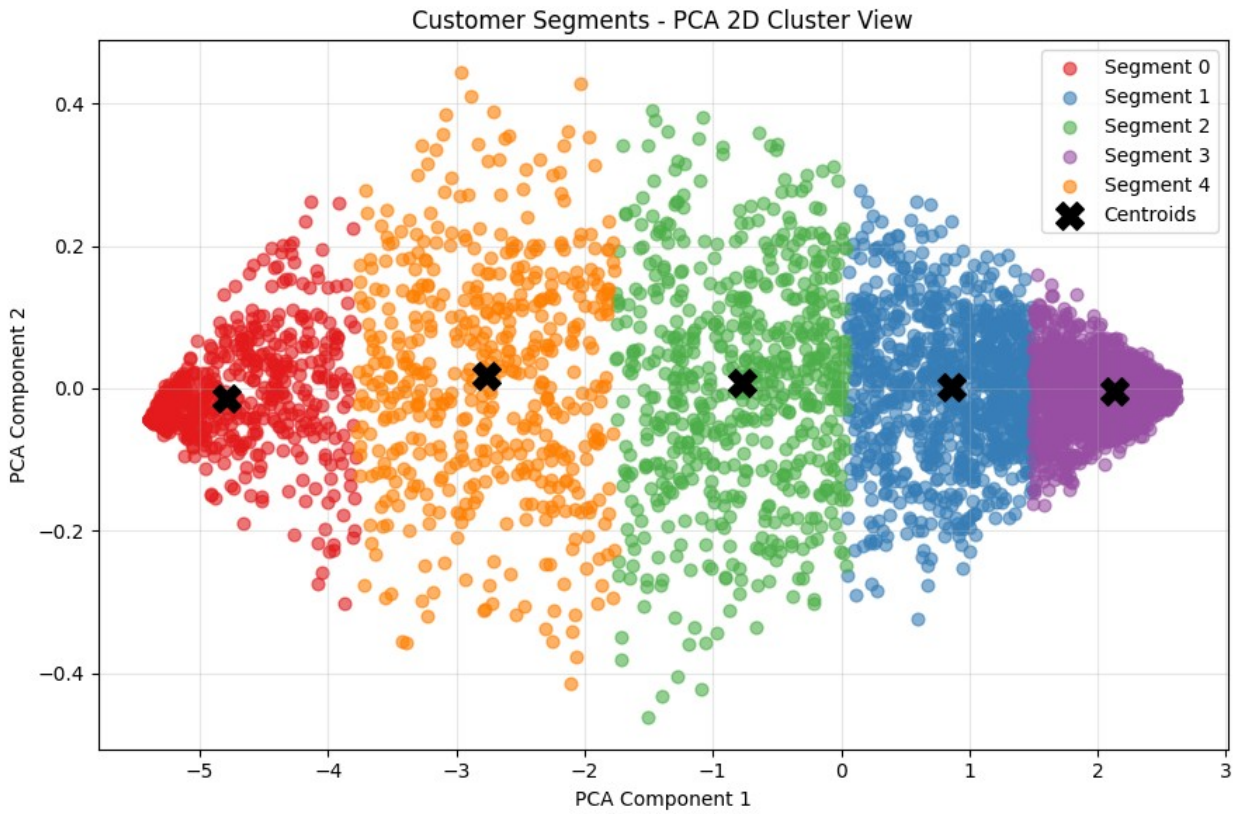
```
1        12347        2
2        12348        3
3        12349        4
4        12350        3
5        12351        3
6        12352        2
7        12353        3
8        12354        2
9        12355        3
```

```python
# ── Summary ──────────────────────────────────────────────────────
print("\n===== SUMMARY =====")
print(f"Total customers analysed : {binary_matrix.shape[0]}")
print(f"Unique products used     : {binary_matrix.shape[1]}")
print(f"RBM hidden units         : {N_HIDDEN}")
print(f"Final Train Loss         : {train_losses[-1]:.4f}")
print(f"Final Test Loss          : {test_loss:.4f}")
print(f"Number of segments       : 5 (K-Means on RBM embeddings)")
```

```
===== SUMMARY =====
Total customers analysed : 5724
Unique products used     : 500
RBM hidden units         : 100
Final Train Loss         : 0.0617
Final Test Loss          : 0.0565
Number of segments       : 5 (K-Means on RBM embeddings)
```

```python
# ── GRAPH 3: PCA - Customer Segments in 2D ────────────────────
from sklearn.decomposition import PCA
import matplotlib.pyplot as plt
import seaborn as sns

pca = PCA(n_components=2)
reduced = pca.fit_transform(embeddings)

plt.figure(figsize=(9, 6))
colors = sns.color_palette("Set1", 5)

for seg in range(5):
    idx = clusters == seg
    plt.scatter(reduced[idx, 0], reduced[idx, 1],
                label=f"Segment {seg}",
                alpha=0.6, s=40, color=colors[seg])

# Plot cluster centers
centers_2d = pca.transform(kmeans.cluster_centers_)
plt.scatter(centers_2d[:, 0], centers_2d[:, 1],
            marker='X', s=200, c='black', label='Centroids', zorder=5)

plt.title("Customer Segments - PCA 2D Cluster View")
```

```
plt.xlabel("PCA Component 1")
plt.ylabel("PCA Component 2")
plt.legend()
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.savefig("/content/drive/MyDrive/cluster_graph.png")
plt.show()

print("□ Cluster graph saved!")
```




```
□ Cluster graph saved!
```